BRADIX Cloud Ingestion Pipeline

All Cloud Sources → NUC → Jetson AI Processing

OVERVIEW

Everything flows into your NUC, gets processed by Jetson locally, and is stored on your D-Link NAS. Nothing leaves your network.

```
Google Photos ──┐
iCloud ─────────┤
OneDrive ───────┤──→ NUC (n8n orchestration) ──→ Jetson (AI processing) ──→
NAS (storage)
Gmail ──────────┤
↓
Ring ───────────┤
BRADIX index
Samsung ────────┘
↓
                                                                    OpenClaw
/ Dashboard
```

---

1. GOOGLE PHOTOS

One-Time Bulk Download (do this first)

1. Go to **takeout.google.com**
2. Select “Google Photos” only
3. Choose “Export once”, .zip format, maximum file size
4. Download all zip files to NAS: `/nas/photos/google/`

Ongoing Sync (gphotos-sync)

```
# Install on NUC
pip3 install gphotos-sync

# Authenticate
gphotos-sync --auth /home/ubuntu/.gphotos /nas/photos/google/

# Schedule daily sync (add to crontab)
0 2 * * * gphotos-sync /nas/photos/google/ >> /var/log/gphotos.log 2>&1
```

Jetson AI Processing

```
# Install face recognition on Jetson
pip3 install face_recognition deepface

# Run evidence tagging script
python3 /home/ubuntu/bradix_agents/tag_photos.py
# Tags: people identified, dates, locations, potential evidence items
```

---

2. iCLOUD (cherylbruder@icloud.com + Andrew's accounts)

Install icloud-photos-downloader

```
# On NUC
pip3 install icloudpd

# Download Cheryl's photos (evidence of her state, communications)
icloudpd --directory /nas/photos/icloud-cheryl \
  --username cherylbruder@icloud.com \
  --password [PASSWORD] \
  --recent 500

# Download Andrew's photos
icloudpd --directory /nas/photos/icloud-andrew \
  --username [ANDREW-ICLOUD-EMAIL] \
  --password [PASSWORD]
```

Note on 2FA: icloudpd handles 2FA — it will prompt for the code the first time. After that, it stores the session cookie.

Ongoing Sync

```
# Add to crontab for daily sync
0 3 * * * icloudpd --directory /nas/photos/icloud-cheryl --username
cherylbruder@icloud.com --password [PASSWORD] --auto-delete >>
/var/log/icloudpd.log 2>&1
```

What to Look For in Cheryl's iCloud:

- Photos showing her state of mind and physical condition over time
- Any communications with PTQ or other institutions
- Evidence of her capacity and daily functioning
- Photos of the property before BHC disposal

---

3. ONEDRIVE — DTMR VOICE MEMO + ALL FILES

Microsoft Graph API Setup

1. Go to **portal.azure.com** → Azure Active Directory → App registrations
2. New registration: “BRADIX-OneDrive”
3. Add permissions: Files.Read.All, offline_access
4. Note your Client ID and Tenant ID
5. Generate a client secret

Download Script

```
pip3 install msal requests

# Save as /home/ubuntu/bradix_agents/onedrive_sync.py
```

```python
import msal, requests, os

CLIENT_ID = "[YOUR-CLIENT-ID]"
TENANT_ID = "[YOUR-TENANT-ID]"
CLIENT_SECRET = "[YOUR-SECRET]"
DOWNLOAD_PATH = "/nas/onedrive/"

app = msal.ConfidentialClientApplication(
    CLIENT_ID,
    authority=f"https://login.microsoftonline.com/{TENANT_ID}",
    client_credential=CLIENT_SECRET
)

token = app.acquire_token_for_client(scopes=
["https://graph.microsoft.com/.default"])
headers = {"Authorization": f"Bearer {token['access_token']}"}

# Get all files
response =
requests.get("https://graph.microsoft.com/v1.0/me/drive/root/children",
headers=headers)
files = response.json()["value"]

for f in files:
    if f.get("file"):
        download_url = f["@microsoft.graph.downloadUrl"]
        filename = f["name"]
        print(f"Downloading: {filename}")
        content = requests.get(download_url).content
        with open(os.path.join(DOWNLOAD_PATH, filename), "wb") as out:
            out.write(content)
```

Auto-Transcribe Voice Memos (Whisper on Jetson)

```
# Install Whisper on Jetson
pip3 install openai-whisper

# Transcribe DTMR voice memo
whisper /nas/onedrive/[DTMR-MEMO-FILENAME].m4a \
  --model medium \
  --language en \
  --output_dir /nas/transcripts/ \
  --output_format txt

# Output: /nas/transcripts/[DTMR-MEMO-FILENAME].txt
```

---

4. GMAIL — ALL 10+ ACCOUNTS

Gmail API Setup (per account)

1. Go to **console.cloud.google.com**
2. Create project “BRADIX-Gmail”
3. Enable Gmail API
4. Create OAuth 2.0 credentials
5. Download credentials.json

Full History Download

```
pip3 install google-auth google-auth-oauthlib google-api-python-client

# Save as /home/ubuntu/bradix_agents/gmail_backup.py
```

```python
from google.oauth2.credentials import Credentials
from googleapiclient.discovery import build
import base64, json, os

def backup_gmail(account_name, credentials_path):
    creds = Credentials.from_authorized_user_file(credentials_path)
    service = build('gmail', 'v1', credentials=creds)

    # Get all messages
    messages = []
    response = service.users().messages().list(userId='me',
maxResults=500).execute()
    messages.extend(response.get('messages', []))

    while 'nextPageToken' in response:
        response = service.users().messages().list(
            userId='me',
            pageToken=response['nextPageToken'],
            maxResults=500
        ).execute()
        messages.extend(response.get('messages', []))

    print(f"{account_name}: {len(messages)} messages found")

    # Download each message
    os.makedirs(f"/nas/email/{account_name}", exist_ok=True)
    for msg in messages:
        full_msg = service.users().messages().get(userId='me',
id=msg['id']).execute()
        with open(f"/nas/email/{account_name}/{msg['id']}.json", 'w') as f:
            json.dump(full_msg, f)
```

n8n Gmail Monitoring Workflow

- Trigger: New email received
- Filter: From institutional domains (sper.qld.gov.au, tmr.qld.gov.au, pt.qld.gov.au, etc.)
- Action: Extract key info → Add to BRADIX case tracker → Send phone alert

---

5. RING CAMERA — DOWNLOAD BEFORE 60-DAY EXPIRY

Install python-ring-doorbell

```
pip3 install ring_doorbell

# Save as /home/ubuntu/bradix_agents/ring_download.py
```

```
import ring_doorbell
import os

ring = ring_doorbell.Ring("[RING-EMAIL]", "[RING-PASSWORD]")
ring.update_data()

for device in ring.video_doorbells + ring.stickup_cams:
    print(f"Device: {device.name}")
    for event in device.history(limit=100):
        filename = f"/nas/ring/{device.name}_{event['created_at']}.mp4"
        if not os.path.exists(filename):
            device.recording_download(event['id'], filename=filename)
            print(f"Downloaded: {filename}")
```

Set Up Permanent Local Recording

- In Ring app: Settings → Video Settings → Video Recording Length → Maximum
- In Home Assistant: Add Ring integration → Enable local snapshot saving
- n8n workflow: Every hour, check for new Ring events and download to NAS

URGENT: Run this script TODAY to preserve the car break-in footage before it expires.

---

6. SAMSUNG FAMILY HUB FRIDGE

Via SmartThings API

```
pip3 install pysmartthings

# Get your SmartThings Personal Access Token from:
# account.smartthings.com → Personal Access Tokens
```

```
import asyncio
import aiohttp
import pysmartthings

async def get_fridge_data():
    async with aiohttp.ClientSession() as session:
        api = pysmartthings.SmartThings(session, "[YOUR-SMARTTHINGS-TOKEN]")
        devices = await api.devices()
        for device in devices:
            if "refrigerator" in device.type.lower() or "Family Hub" in
device.label:
                status = await device.status.refresh()
                print(f"Fridge status: {status.values}")

asyncio.run(get_fridge_data())
```

Via USB (for local data)

1. Insert USB drive into Family Hub USB port (usually on the side)
2. On the fridge touchscreen: Apps → My Files → USB Storage
3. Copy any photos, notes, or shopping lists to USB
4. Plug USB into NUC and copy to `/nas/samsung-fridge/`

---

7. iPHONE BACKUP ANALYSIS

Via libimobiledevice (USB connection)

```
# Install on NUC
sudo apt install -y libimobiledevice-utils ifuse

# Connect iPhone via USB
ideviceinfo  # Verify connection
idevicebackup2 backup --full /nas/iphone-backup/

# Mount backup for analysis
ifuse --documents /nas/iphone-mount/
```

Extract Key Data from Backup

```
pip3 install iphone-backup-decrypt

python3 -c "
from iphone_backup_decrypt import EncryptedBackup, RelativePath
backup = EncryptedBackup(backup_directory='/nas/iphone-backup/',
passphrase='[BACKUP-PASSWORD]')
backup.extract_file(relative_path=RelativePath.SMS,
output_filename='/nas/iphone-data/sms.db')
backup.extract_file(relative_path=RelativePath.CALL_HISTORY,
output_filename='/nas/iphone-data/calls.db')
"
```

MASTER INGESTION SCRIPT

Save as `/home/ubuntu/bradix_agents/ingest_all.sh`:

```bash
#!/bin/bash
echo "=== BRADIX FULL CLOUD INGESTION ==="
echo "Started: $(date)"

echo "1. Syncing Google Photos..."
gphotos-sync /nas/photos/google/

echo "2. Syncing iCloud (Cheryl)..."
icloudpd --directory /nas/photos/icloud-cheryl --username
cherylbruder@icloud.com

echo "3. Downloading OneDrive..."
python3 /home/ubuntu/bradix_agents/onedrive_sync.py

echo "4. Downloading Ring footage..."
python3 /home/ubuntu/bradix_agents/ring_download.py

echo "5. Backing up Gmail accounts..."
python3 /home/ubuntu/bradix_agents/gmail_backup.py

echo "=== INGESTION COMPLETE: $(date) ==="
```

```bash
chmod +x /home/ubuntu/bradix_agents/ingest_all.sh
# Run now:
./ingest_all.sh
```

PRIORITY ORDER — DO THESE FIRST

1. **Ring footage download** — 60-day expiry, car break-in evidence. Run ring_download.py TODAY.
2. **OneDrive download** — Get the DTMR voice memo. Run onedrive_sync.py TODAY.
3. **iCloud download** — Cheryl's account may have evidence. Set up icloudpd this week.
4. **Gmail full backup** — All 10+ accounts. Run gmail_backup.py this week.
5. **Google Photos** — Use Takeout first (bulk), then set up ongoing sync.
6. **Samsung fridge** — USB extraction when convenient.